

Manual del Programador

Arquitectura, configuración y despliegue de Trips SV

Sistema	Trips SV - CMS de turismo de El Salvador
Audiencia	Desarrolladores, administradores técnicos y responsables de despliegue.
Alcance	Describe la arquitectura MVC por capas, el acceso seguro a SQL Server, la interfaz, las pruebas y los procedimientos para ejecutar, publicar y mantener el sistema.
Versión	1.0 Revisión funcional y técnica local

Documento preparado a partir de la revisión del código fuente y del recorrido de la aplicación en modo Development. Las figuras muestran referencias visuales de las pantallas observadas y de sus controles principales.

Contenido

1. Resumen técnico
2. Arquitectura del sistema
3. Tecnologías utilizadas
4. Estructura de la base de código
5. Modelo de datos
6. Acceso a datos y persistencia
7. Interfaces de usuario, responsividad y accesibilidad
8. Seguridad y manejo de errores
9. Configuración del entorno
10. Despliegue
11. Pruebas automatizadas
12. Mantenimiento y extensión

1 Resumen técnico

Trips SV es un CMS web para la publicación y exploración de destinos turísticos de El Salvador. La versión revisada está construida con ASP.NET Core MVC sobre .NET 10, Entity Framework Core Code First, SQL Server y ASP.NET Core Identity. El catálogo cargado por el script de datos contiene 9 categorías, 27 sitios y comentarios de la comunidad.

Elemento	Implementación verificada
Framework	.NET 10 / ASP.NET Core MVC con Razor
Persistencia	Entity Framework Core 10 con proveedor SQL Server y migraciones Code First
Autenticación	ASP.NET Core Identity con roles Administrador y Usuario
Interfaz	Bootstrap local, Bootstrap Icons, CSS propio, jQuery Validation Unobtrusive y CKEditor
Seguridad de contenido	HtmlSanitizer para el contenido enriquecido y validación de imágenes
Pruebas	Proyecto TripSV. Pruebas con xUnit y EF Core SQLite en memoria; el repositorio declara 80 casos

2 Arquitectura del sistema

Tipo de arquitectura: MVC con N-capas lógicas dentro de un único proyecto web. MVC separa Modelo, Vista y Controlador; la separación por capas agrega una capa de servicios de negocio entre los controladores y el acceso a datos. No es MVVM: las vistas Razor se renderizan en el servidor y los ViewModels transportan datos y validación, pero no existe un enlace bidireccional reactivo propio de una aplicación SPA.

Capa	Responsabilidad	Componentes principales
Presentación	Renderiza páginas, formularios y mensajes; aplica validación de entrada y autorización.	Controllers, Areas/Administracion, Views Razor, ViewModels, wwwroot
Negocio	Centraliza reglas de búsqueda, comentarios, calificaciones, favoritos, itinerarios y resultados de operación.	Interfaces e implementaciones en Servicios/
Datos	Mapea entidades, relaciones, índices y restricciones; persiste cambios y ejecuta migraciones.	Modelos/, ContextoTripSV, Configuraciones/, Migraciones/
Infraestructura	Compone el contenedor, Identity, middleware, archivos estáticos y configuración.	Program.cs, appsettings.json, ASP.NET Core Identity

Solicitud HTTP -> Controller -> Servicio (interfaz) -> ContextoTripSV -> SQL Server
-> ViewModel -> Razor View -> Respuesta HTML

2.1 Justificación de la elección

- Renderizado del lado del servidor: el catálogo y las páginas públicas reciben HTML completo, con una carga de JavaScript reducida y una navegación apropiada para un sitio de contenidos.
- Formularios con validación integrada: MVC combina model binding, DataAnnotations, validación unobtrusive y tokens antifalsificación para las operaciones de cuenta, reseñas y administración.
- Reglas testeables: los controladores son delgados y los servicios pueden probarse con un ContextoTripSV sobre SQLite en memoria sin levantar el servidor web.
- Separación de responsabilidades: la persistencia queda concentrada en EF Core, las reglas en Servicios y el módulo administrativo en un Area protegida por rol.
- Proporción con el proyecto: un monolito modular evita el costo de varios despliegues y comunicaciones de microservicios para un dominio pequeño/mediano, pero mantiene límites claros para evolucionar el sistema.

2.2 Flujo de una operación

1. El navegador envía una solicitud, por ejemplo POST /Puntuaciones/Calificar, con el token antifalsificación.
2. El controlador comprueba la sesión mediante [Authorize] y valida el modelo recibido.
3. El controlador invoca IPuntuacionesServicio; el servicio comprueba el rango 1–5, crea o actualiza la calificación y recalcula el promedio.
4. El servicio devuelve Resultado.Ok o Resultado.Error; el controlador guarda el mensaje en TempData y redirige al detalle usando Post/Redirect/Get.

Patrón o principio	Aplicación en Trips SV
Inyección de dependencias	Program.cs registra interfaces; controladores y servicios reciben sus dependencias por constructor.
Unidad de trabajo	ContextoTripSV y sus DbSet coordinan las lecturas y escrituras de una solicitud.
ViewModel	Las pantallas que combinan datos usan modelos de vista específicos, sin depender siempre de las entidades.
Result	Resultado comunica errores esperables sin usar excepciones como flujo normal.
SOLID	Interfaces pequeñas por módulo; validación de imágenes y saneamiento HTML en servicios separados.

3 Tecnologías utilizadas

Componente	Tecnología	Uso
Plataforma	.NET 10 / ASP.NET Core	Servidor web y ciclo de vida de la aplicación.
Presentación	MVC, Razor, Bootstrap 5, Bootstrap Icons	Vistas, navegación, grid responsive e iconografía.

Datos	EF Core 10 / SQL Server	Mapeo ORM, consultas LINQ, restricciones y migraciones.
Identidad	ASP.NET Core Identity	Usuarios, contraseñas, cookies, roles y tokens.
Contenido	HtmlSanitizer 9.x / CKEditor	Edición y saneamiento de información enriquecida.
Cliente	jQuery Validation Unobtrusive	Validación de formularios en el navegador.
Pruebas	xUnit / EF Core SQLite	Pruebas de servicios, validación, saneamiento y autorización.

4 Estructura de la base de código

TripSV.slnx	
└─ TripSV/	
└─ Program.cs	Composición, middleware y rutas
└─ Modelos/	Entidades y roles
└─ Datos/	DbContext, Fluent API, migraciones y sembrador
└─ Servicios/	Interfaces y reglas de negocio
└─ ViewModels/	Modelos para formularios y páginas
└─ Controllers/	Flujos públicos y de usuario
└─ Areas/Administracion/	CRUD y moderación protegidos
└─ Views/	Vistas Razor y parciales
└─ wwwroot/	CSS, JS, imágenes y librerías
└─ TripSV.Pruebas/	Pruebas automatizadas
└─ Herramientas/	Utilidades de datos/documentación
└─ Documentacion/	Manuales, fuentes y capturas
└─ Ejecutable/	Publicación Windows x64

Módulo	Responsabilidad
Controllers	Inicio, categorías, catálogo, cuenta, comentarios, puntuaciones, favoritos e itinerarios.
Areas/Administracion	Categorías, sitios y moderación; las acciones exigen el rol Administrador.
Servicios	Reglas del dominio, validaciones, consultas y guardado con manejo de conflictos.
Datos	ContextoTripSV, configuraciones IEntityTypeConfiguration, migraciones y seed de Identity.
Views y ViewModels	Interfaz pública, formularios, mensajes, tablas, tarjetas y pantallas administrativas.
TripSV.Pruebas	Casos de xUnit para servicios, validación de imágenes, saneamiento de HTML y seguridad.

El menú Administrar se oculta para otros roles, pero la garantía real está en la autorización del servidor. Las vistas parciales _TarjetaSitio, _TarjetaCategoria, _Mensajes y los componentes de estrellas permiten una interfaz consistente.

5 Modelo de datos

ContextoTripSV hereda de IdentityDbContext<Usuario>, aplica las configuraciones de todo el ensamblado con ApplyConfigurationsFromAssembly y expone DbSet para el dominio de turismo.

Tabla / entidad	Reglas y relaciones importantes
categorias / Categoria	Nombre único; una categoría tiene muchos sitios y no se elimina si aún tiene destinos.
sitios / Sitio	Nombre único; pertenece a una categoría; guarda imagen, información enriquecida, promedio y total de puntuaciones.
comentarios / Comentario	Reseñas y respuestas; puede ocultarse; respuestas enlazadas mediante respuesta_a_id.
puntuaciones / Puntuacion	Valor de 1 a 5; índice único por sitio y usuario; CHECK de base de datos.
favoritos / Favorito	Índice único por usuario y sitio; borrado en cascada con usuario o sitio.
itinerarios / Itinerario	Pertenece a un usuario; CHECK de 1 a 15 días.
visitas / Visita	Destino por día y orden; índice único por itinerario, día y sitio.
Identity	usuarios, roles, usuarios_roles y tablas de claims, logins y tokens con nombres en español.

Al eliminar	Comportamiento
Destino	Cascada para comentarios, puntuaciones, favoritos y visitas.
Usuario	Cascada para favoritos e itinerarios; comentarios y calificaciones conservan el nombre del autor.
Itinerario	Cascada para sus visitas.
Comentario	El servicio elimina primero sus respuestas.
Categoría	Restrict: se impide mientras existan sitios asociados.

6 Acceso a datos y persistencia

6.1 Entity Framework Core Code First

El esquema se define en C# mediante entidades y clases de configuración Fluent API. ContextoTripSV traduce las consultas LINQ a SQL Server, mientras que las migraciones materializan cambios de esquema. SembradorDatos ejecuta Database.MigrateAsync al iniciar y crea los roles y usuarios iniciales si no existen.

```
dotnet ef migrations add NombreDeLaMigracion --project TripSV --output-dir Datos/Migraciones
dotnet ef database update --project TripSV
```

Las clases de Configuraciones definen nombres snake_case, tipos de columna, longitudes, índices únicos, CHECK constraints y comportamientos de borrado. ExtensionesContexto.GuardarSinConflictoAsync encapsula SaveChangesAsync, limpia el ChangeTracker cuando ocurre un DbUpdateException y devuelve un resultado de conflicto para que el servicio informe al usuario.

6.2 Cadena de conexión segura

Program.cs obtiene la clave ConnectionStrings:ConexionTripSV mediante GetConnectionString y falla temprano si no existe. La configuración local actual usa autenticación integrada de Windows contra LocalDB; no incluye usuario ni contraseña.

```
"ConexionTripSV":
"Server=(localdb)\mssqllocaldb;Database=TripSV;Trusted_Connection=True;TrustServerCertificate=True;MultipleActiveResultSets=True"
```

Entorno	Mecanismo recomendado	Ejemplo
Desarrollo	User Secrets fuera del repositorio	dotnet user-secrets set "ConnectionStrings:ConexionTripSV" "Server=...;" --project TripSV
CI/CD	Variable protegida del pipeline	ConnectionStrings__ConexionTripSV
Producción	Secret manager o variable de entorno	Server=...;User Id=...;Password=...;Encrypt=True

- Nunca versionar contraseñas de SQL Server, tokens o credenciales de demostración.
- TrustServerCertificate=True es aceptable solo para desarrollo local; en producción usar Encrypt=True y un certificado válido.
- El usuario de producción debe tener permisos mínimos y la cadena debe inyectarse desde el ambiente, no editarse en el binario publicado.

6.3 Uso eficiente de la persistencia

- Los listados usan proyecciones que omiten la columna binaria de imagen; SinImagen selecciona solo los campos necesarios y la imagen se obtiene por /Sitios/Imagen/{id}.
- Las imágenes del endpoint dedicado tienen ResponseCache de una hora; los archivos estáticos tienen Cache-Control de siete días.
- Los conteos, filtros y ordenamientos se ejecutan en SQL mediante LINQ; la búsqueda admite hasta cinco términos y usa una collation tolerante a tildes y mayúsculas.

- Los índices únicos también hacen cumplir reglas de negocio para nombres, favoritos, puntuaciones y visitas.
- Las lecturas que no necesitan seguimiento usan proyecciones o AsNoTracking; las operaciones de actualización mantienen el seguimiento solo de la entidad que se modifica.
- La calificación promedio y el total de votos se almacenan en Sitio y se recalculan únicamente cuando cambia una puntuación.

6.4 Datos iniciales

El catálogo no se carga desde SembradorDatos. TripSV/Datos/Migracion/datos-mysql.sql contiene categorías, destinos, imágenes, reseñas y puntuaciones, y debe ejecutarse una vez después de crear el esquema:

```
sqlcmd -S "(localdb)\MSSQLLocalDB" -d TripSV -E -i "TripSV\Datos\Migracion\datos-mysql.sql" -b
```

7 Interfaces de usuario, responsividad y accesibilidad

La interfaz es una aplicación server-rendered con Razor, Bootstrap local y un sistema de estilos propio en wwwroot/css/site.css. La navegación, tarjetas, formularios y mensajes comparten componentes visuales para que las rutas públicas, de usuario y de administración se perciban como un mismo producto.

Criterio	Implementación actual	Criterio de mantenimiento
Intuitiva	Barra con Inicio, Categorías y Destinos; buscador global; menús de usuario y administración; mensajes de éxito/error.	Mantener nombres de acción claros, estado activo y acciones destructivas diferenciadas.
Responsiva	Bootstrap con container/row/col, breakpoints md/lg, tarjetas que se apilan y navbar colapsable en móvil.	Probar 320, 768 y 1280 px en cada pantalla nueva.
Accesible	lang=es, enlace para saltar al contenido, labels asociados, aria-label/aria-controls, estados de validación y alt en imágenes.	Verificar teclado, foco visible, contraste y que la información no dependa solo del color.
Cohesiva	Variables de color, tipografías Poppins/Inter, botones y tarjetas reutilizables, parciales Razor.	Reutilizar site.css y las parciales antes de crear estilos aislados.

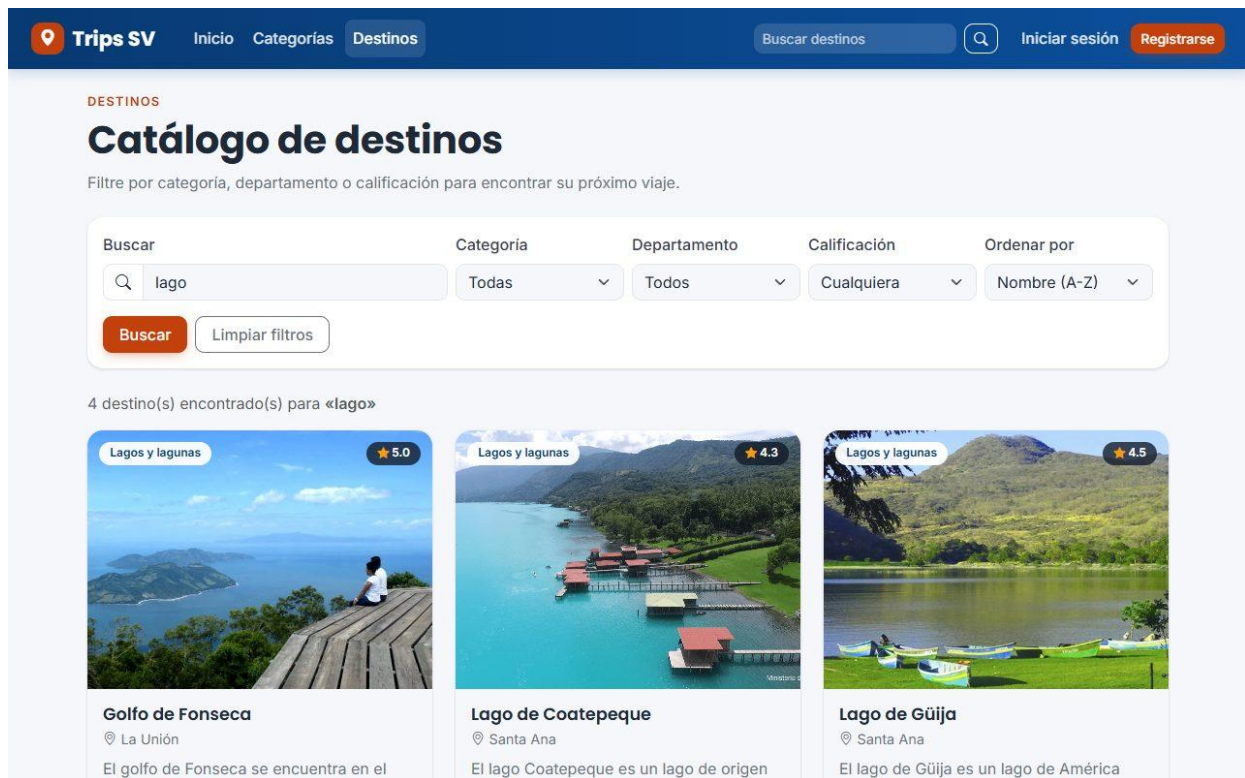


Figura 1. Captura actual del catálogo rediseñado con búsqueda, filtros y ordenamiento.



Figura 2. La portada reorganiza el menú y el contenido para una pantalla móvil.

Puntos de atención para nuevas vistas: conservar encabezados jerárquicos, asociar cada input con un label, proporcionar mensajes de validación comprensibles, usar alt descriptivo en imágenes informativas y aria-

hidden en iconos decorativos. El enlace externo de Google Fonts y Bootstrap Icons debe evaluarse si se despliega en un ambiente sin salida a Internet.

8 Seguridad y manejo de errores

Riesgo o requisito	Medida implementada
Autenticación	Identity; mínimo 8 caracteres, dígito y minúscula; bloqueo de 10 minutos después de 5 intentos fallidos.
Autorización	Roles Usuario y Administrador; [Authorize] en módulos privados y autorización por rol en Areas/Administracion.
Sesión	Cookie de Identity con expiración deslizante de 90 minutos.
CSRF	Tokens antifalsificación en formularios POST y validación en acciones de escritura.
XSS	Razor codifica la salida; SanitizadorHtml elimina scripts, eventos y URLs no permitidas de iframes.
Imágenes	JPG/JPEG/PNG, máximo 5 MB, tipo validado antes de leer bytes.
Headers	nosniff, SAMEORIGIN, Referrer-Policy y Permissions-Policy; HTTPS redirection fuera de Development.
Errores	UseExceptionHandler en producción, página de estado para 404/403 y no-store en páginas de error.

El contenido enriquecido se sanea al crear/actualizar y se vuelve a limpiar antes de mostrarlo. Solo se permiten iframes de Google Maps y YouTube con los atributos admitidos. Las excepciones de infraestructura deben observarse en logs, sin mostrar detalles internos al visitante.

9 Configuración del entorno

Requisitos mínimos: SDK .NET 10, una instancia SQL Server/LocalDB, editor con soporte C# y Razor, y acceso a NuGet durante el primer restore. La solución contiene TripSV/TripSV.csproj y TripSV.Pruebas/TripSV.Pruebas.csproj dentro de TripSV.slnx.

1. Abrir una terminal en la raíz del repositorio.
2. Configurar ConexionTripSV en User Secrets o en appsettings.json solo para desarrollo local.
3. Restaurar y compilar con dotnet build TripSV.slnx.
4. Crear/actualizar el esquema con dotnet ef database update --project TripSV.
5. Cargar el catálogo con el script T-SQL si la base está vacía.
6. Ejecutar dotnet run --project TripSV --launch-profile http y abrir http://localhost:5255.

Usuario de desarrollo	Contraseña	Rol
administrador	Administrador123\$	Administrador
visitante	Visitante123\$	Usuario

Las credenciales anteriores son únicamente para demostración local. Deben cambiarse o eliminarse antes de publicar.

10 Despliegue

Se recomienda separar la publicación de la aplicación, la preparación de SQL Server y la carga de datos. Aunque el arranque aplica migraciones pendientes, en producción conviene ejecutar migraciones desde un pipeline con trazabilidad y respaldo previo.

1. Crear la base de datos y un usuario de SQL Server con permisos mínimos.
2. Definir la cadena de producción mediante secret manager o ConnectionStrings__ConexionTripSV.
3. Publicar: `dotnet publish TripSV/TripSV.csproj -c Release -o ./publish`.
4. Ejecutar `dotnet ef database update --project TripSV` desde el proceso controlado de despliegue.
5. Cargar el catálogo inicial si el ambiente lo requiere, sin repetirlo sobre una base ya poblada.
6. Hospedar detrás de IIS o reverse proxy con HTTPS, certificado válido, límites de carga y logs configurados.
7. Persistir las claves de Data Protection en un directorio protegido o almacenamiento compartido si hay más de una instancia.
8. Verificar las rutas públicas, login, autorización administrativa, imágenes, mensajes de error y respaldos.

```
dotnet publish TripSV/TripSV.csproj -c Release -o ./publish
dotnet TripSV.dll
```

El repositorio incluye Ejecutable/TripSV.zip como publicación Windows x64. Requiere ASP.NET Core Runtime 10, SQL Server y un appsettings/variable de entorno configurado para el ambiente.

11 Pruebas automatizadas

TripSV.Pruebas contiene 80 casos xUnit para calificaciones, comentarios y moderación, favoritos, itinerarios, categorías y destinos, validación de imágenes, saneamiento de HTML y reglas de seguridad de controladores. Los servicios usan una base SQLite en memoria para conservar relaciones, índices únicos y claves foráneas durante las pruebas.

Suite	Cobertura
CategoriasYSitiosServicioPruebas	Búsqueda, categorías, destinos, duplicados y eliminaciones.
ComentariosServicioPruebas	Publicación, respuestas, ocultamiento, eliminación y límites.
PuntuacionesServicioPruebas	Rango 1–5, actualización y recálculo del promedio.

FavoritosServicioPruebas	Agregar, quitar, duplicados y listado por usuario.
ItinerariosServicioPruebas	Días, orden, movimientos, máximo por día y pertenencia.
Seguridad/ValidacionYSanitizacion	Antiforgery, roles, imágenes, HTML permitido y URLs de iframe.

```
dotnet test
```

Antes de integrar cambios, ejecutar la suite completa, compilar en Release y comprobar también una base SQL Server con las migraciones. Las pruebas de SQLite no sustituyen la verificación del proveedor SQL Server en el ambiente destino.

12 Mantenimiento y extensión

Tema	Recomendación
Secretos	Mover credenciales, cadenas y claves a secretos del ambiente; no versionar valores reales.
Migraciones	Crear una migración por cambio de esquema y revisarla antes de aplicarla en producción.
Backups	Respaldar SQL Server y validar restauración antes de migraciones destructivas.
Data Protection	Persistir claves en servidores con más de una instancia para no invalidar cookies.
Contenido	Conservar el saneamiento HTML y ampliar la lista de orígenes solo después de revisar seguridad.
Imágenes	Mantener el límite de 5 MB, formatos permitidos y endpoint cacheado separado.
Pruebas	Agregar casos de servicio y de autorización antes de incorporar un módulo nuevo.

Para agregar un módulo: crear entidad y configuración Fluent API, agregar DbSet y migración, implementar interfaz/servicio, registrarlo en Program.cs, crear controlador y ViewModels, construir vistas accesibles y añadir pruebas en TripSV.Pruebas. Mantener el Area para administración y [ValidateAntiForgeryToken] en toda acción POST.

Trips SV

Inicio
Categorías
Destinos
Administrar

administrador
Admin

Moderación de comentarios

86
Visibles

1
Ocultos

☐ Mostrar solo ocultos

ID	Usuario	Sitio	Tipo	Comentario	Fecha	Estado	Acciones
171	administrador	El Tazumal	Respuesta a #170	Gracias por su reseña, nos alegra que lo haya disfrutado.	28/09/2026 07:17 p. m.	Visible	Ocultar Eliminar
170	visitante	El Tazumal	Reseña	Excelente para aprender sobre la cultura maya. Recomiendo ir temprano para evitar el calor.	28/09/2026 07:17 p. m.	Visible	Ocultar Eliminar
168	visitante	Ruta de Las Flores	Reseña	Muy bonito lugar muy recomendado para turismo	19/08/2026 08:11 p. m.	Visible	Ocultar Eliminar
166	visitante	Ruta de Las Flores	Reseña	Lugar muy bueno tuve una gran experiencia solo que creo que fue un poco sucio	18/08/2026 09:11 p. m.	Oculto	Mostrar Eliminar
163	Lucas	Costa del Sol	Reseña	Me ha servido mucho la ayuda. Se los recomiendo, amigos.	21/08/2020 06:31 p. m.	Visible	Ocultar Eliminar
161	Eduardo	Guatajiagua	Reseña	Excellent place.	21/08/2020 11:46 a. m.	Visible	Ocultar Eliminar

Figura 3. Captura actual del panel de moderación rediseñado.

Checklist de entrega: compilar, ejecutar pruebas, aplicar migraciones en una base de prueba, validar secretos, comprobar HTTPS, revisar autorización por rol, verificar imágenes y contenido enriquecido, y confirmar que el catálogo esté cargado.